

SOFTWARE DESIGN DOCUMENT (SDD)

EV CHARGING NETWORK APP (ECNA)

T.C. KONYA FOOD AND AGRICULTURE UNIVERSITY

Faculty of Engineering and Architecture

Department of Computer Engineering

Project Team:

- Emre Erkut – 222010020156
- Keramettin Kutnu – 222010020155
- Yunus Emre Kahraman – 222010020157

Supervisor: Prof. Dr. Meltem Huri Baturay

Document Version: 1.0

Date: November 2025

TABLE OF CONTENTS

- [1. Introduction](#)
- [2. System Overview](#)
- [3. System Architecture](#)
- [4. Detailed Design](#)
- [5. Database Design](#)
- [6. API Design](#)
- [7. User Interface Design](#)
- [8. Security Design](#)
- [9. IoT Integration Design](#)
- [10. Performance Requirements](#)
- [11. Deployment Architecture](#)
- [12. Testing Strategy](#)
- [13. References](#)

1. INTRODUCTION

1.1 Purpose

This Software Design Document (SDD) provides a comprehensive description of the design for the EV Charging Network App (ECNA). The document specifies the architectural design, component structure, database schema, API interfaces, security mechanisms, and integration strategies for the system.

1.2 Scope

The ECNA system is a mobile application platform that enables electric vehicle users to locate, reserve, and pay for charging stations efficiently. The system integrates IoT-enabled real-time data collection with cloud-based services to optimize station availability and reduce waiting times.

This document covers:

- System architecture and component design

- Frontend and backend module specifications
- Database schema and data models
- RESTful API endpoints
- User interface design principles
- Security mechanisms and authentication
- IoT integration via MQTT protocol
- Payment gateway integration
- Deployment and scalability considerations

1.3 Definitions, Acronyms, and Abbreviations

- **ECNA**: EV Charging Network App
- **EV**: Electric Vehicle
- **IoT**: Internet of Things
- **MQTT**: Message Queuing Telemetry Transport
- **API**: Application Programming Interface
- **JWT**: JSON Web Token
- **FCM**: Firebase Cloud Messaging
- **OAuth**: Open Authorization
- **REST**: Representational State Transfer
- **SDK**: Software Development Kit
- **SDD**: Software Design Document

1.4 References

- International Energy Agency (IEA). (2023). Global EV Outlook 2023: Catching up with climate ambitions. Paris: IEA Publications.
 - Azzopardi, B., & Gan, C. K. (2022). Smart Charging Infrastructure for Electric Vehicles: Integration with Smart Grids and Renewable Energy Systems. Renewable and Sustainable Energy Reviews, 161, 112357.
 - Shukla, A., & Jain, S. (2021). Cloud-Based IoT Architecture for Electric Vehicle Charging Network Management. IEEE Access, 9, 77154–77165.
 - Firebase Documentation: <https://firebase.google.com/docs>
 - React Native Documentation: <https://reactnative.dev/docs/getting-started>
 - Stripe API Documentation: <https://stripe.com/docs/api>
-

2. SYSTEM OVERVIEW

2.1 System Purpose

The EV Charging Network App aims to address the challenges in EV infrastructure management by providing:

1. **Real-time Station Discovery**: Users can locate nearby charging stations with live availability status
2. **Reservation Management**: Advanced booking system to reduce waiting times
3. **Secure Payment Processing**: Integrated payment gateway for seamless transactions
4. **IoT Integration**: Real-time data synchronization from charging station sensors
5. **Administrative Dashboard**: Analytics and monitoring tools for station operators

2.2 System Context

The ECNA system operates within a larger ecosystem comprising:

- **Mobile Applications** (iOS/Android): User-facing applications built with React Native
- **Backend API Server**: Node.js/Express.js RESTful API
- **Cloud Database**: Firebase Firestore for data storage

- **Authentication Service:** Firebase Authentication
- **Payment Gateway:** Stripe payment processing
- **MQTT Broker:** Real-time IoT communication
- **IoT Devices:** Charging station sensors and controllers
- **External Services:** Google Maps API, Google Places API

2.3 User Classes and Characteristics

The system serves two primary user classes:

- 1. **EV Users**
 - Primary users seeking charging services
 - Require intuitive mobile interface
 - Need real-time information and quick reservation process
 - Expect secure payment processing
- 2. **Administrators**
 - Station operators and network managers
 - Require analytics and monitoring capabilities
 - Need to manage station data and availability
 - Monitor system performance and user statistics

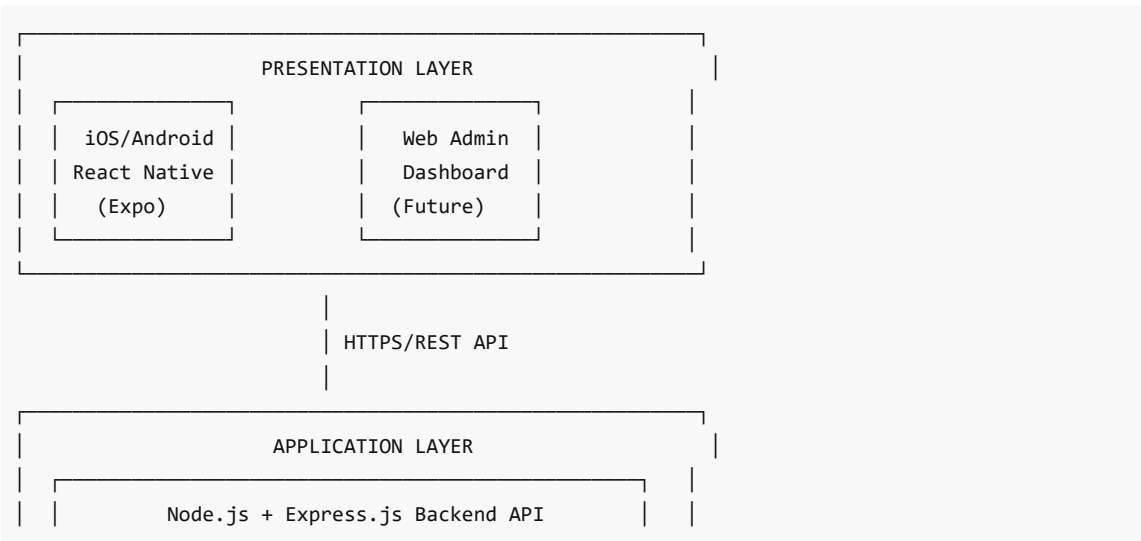
2.4 Operating Environment

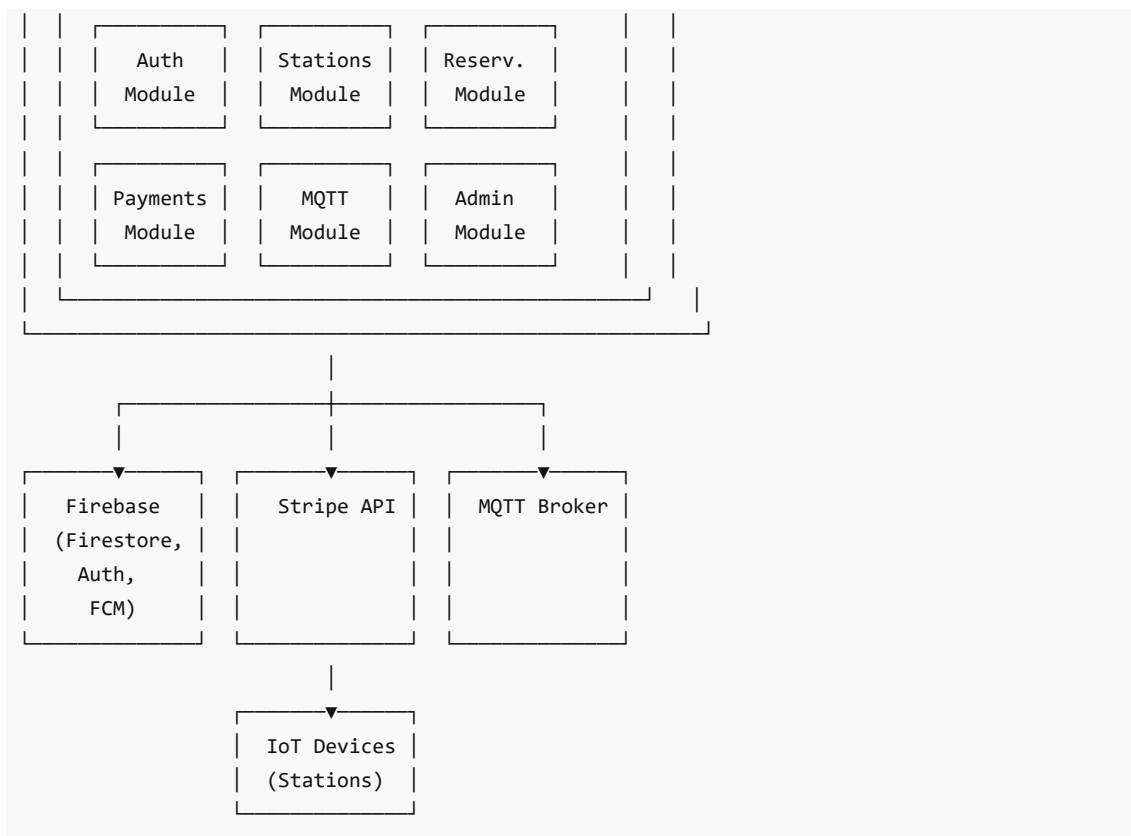
- **Mobile Platforms:** iOS 13.0+, Android 8.0+
- **Backend Server:** Node.js 16+, Express.js 4.18+
- **Cloud Platform:** Firebase (Firestore, Authentication, Cloud Messaging)
- **Payment Processing:** Stripe API
- **IoT Protocol:** MQTT 3.1.1+
- **Maps Service:** Google Maps Platform

3. SYSTEM ARCHITECTURE

3.1 High-Level Architecture

The ECNA system follows a three-tier architecture pattern:





3.2 Architectural Patterns

The system employs the following architectural patterns:

1. **MVC (Model-View-Controller)**: Frontend separates presentation (Views/Screens) from business logic (Services)
2. **RESTful API**: Backend follows REST principles for stateless communication
3. **Microservices-inspired**: Modular backend structure with separate route handlers
4. **Event-Driven Architecture**: MQTT-based real-time updates using publish-subscribe pattern
5. **Layered Architecture**: Clear separation between presentation, application, and data layers

3.3 Component Architecture

3.3.1 Frontend Components

The React Native frontend is organized into the following modules:

```
frontend/  
├─ App.tsx           # Main application entry point  
├─ src/  
│  └─ screens/       # Presentation Layer  
│     ├── LoginScreen.tsx  
│     ├── RegisterScreen.tsx  
│     ├── HomeScreen.tsx  
│     ├── MapScreen.tsx  
│     ├── StationDetailsScreen.tsx  
│     ├── ReservationScreen.tsx  
│     └── PaymentScreen.tsx
```

```

| |   └─ ProfileScreen.tsx
| |   └─ services/           # Business Logic Layer
| |       └─ AuthService.tsx
| |       └─ FirebaseService.ts
| |       └─ MQTTService.ts
| |       └─ PaymentService.ts
| |       └─ NotificationService.ts
| └─ types/                 # Data Models
|     └─ index.ts
| └─ utils/                 # Utility Functions
|     └─ helpers.ts
| └─ config.ts             # Configuration

```

3.3.2 Backend Components

The Node.js backend is structured as follows:

```

backend/
├─ src/
|   └─ server.js           # Express server setup
|   └─ routes/             # Route Handlers
|       └─ auth.js
|       └─ stations.js
|       └─ reservations.js
|       └─ payments.js
|       └─ mqtt.js
|   └─ middleware/         # Middleware Functions
|       └─ auth.js
|   └─ config/             # Configuration
|       └─ firebase.js
|   └─ utils/              # Utility Functions
|       └─ errors.js

```

4. DETAILED DESIGN

4.1 Frontend Module Design

4.1.1 Authentication Module

Purpose: Handle user authentication and session management

Components:

- `AuthService.tsx` : Authentication service wrapper
- `LoginScreen.tsx` : User login interface
- `RegisterScreen.tsx` : User registration interface

Key Functions:

- `register(email, password, displayName)` : Create new user account
- `login(email, password)` : Authenticate user and create session
- `logout()` : Terminate user session
- `getCurrentUser()` : Retrieve current authenticated user

Dependencies: Firebase Authentication SDK

4.1.2 Station Discovery Module

Purpose: Enable users to find and view charging stations

Components:

- `HomeScreen.tsx` : Station list view
- `MapScreen.tsx` : Interactive map with station markers
- `StationDetailsScreen.tsx` : Detailed station information
- `FirebaseService.ts` : Database operations

Key Functions:

- `getNearbyStations(latitude, longitude, radius)` : Query stations by location
- `getStationById(id)` : Retrieve station details
- `subscribeToStationUpdates(stationId, callback)` : Real-time status updates

Dependencies: Firebase Firestore, React Native Maps, Google Places API

4.1.3 Reservation Module

Purpose: Manage charging station reservations

Components:

- `ReservationScreen.tsx` : Create and manage reservations
- `FirebaseService.ts` : Reservation data operations

Key Functions:

- `createReservation(stationId, connectorId, startTime, endTime)` : Create new reservation
- `getUserReservations(userId)` : Retrieve user's reservations
- `updateReservationStatus(reservationId, status)` : Update reservation state
- `cancelReservation(reservationId)` : Cancel existing reservation

Dependencies: Firebase Firestore, Backend API

4.1.4 Payment Module

Purpose: Process secure payment transactions

Components:

- `PaymentScreen.tsx` : Payment interface
- `PaymentService.ts` : Stripe integration

Key Functions:

- `createPaymentIntent(amount, currency)` : Initialize payment
- `confirmPayment(paymentIntentId, paymentMethod)` : Complete transaction
- `getPaymentHistory(userId)` : Retrieve transaction history

Dependencies: Stripe React Native SDK, Backend API

4.1.5 Real-time Updates Module

Purpose: Synchronize real-time data from IoT devices

Components:

- `MQTTService.ts` : MQTT client implementation
- `NotificationService.ts` : Push notification handler

Key Functions:

- `connect(brokerUrl, options)` : Establish MQTT connection
- `subscribe(topic, callback)` : Subscribe to station updates
- `publish(topic, message)` : Publish commands to devices
- `handleNotification(payload)` : Process FCM notifications

Dependencies: MQTT.js, Firebase Cloud Messaging

4.2 Backend Module Design

4.2.1 Authentication Module

Purpose: Verify user identity and manage authorization

Components:

- `routes/auth.js` : Authentication endpoints
- `middleware/auth.js` : JWT verification middleware

Endpoints:

- `POST /api/auth/register` : Register new user
- `GET /api/auth/me` : Get current user profile
- `PUT /api/auth/me` : Update user profile

Key Functions:

- `verifyToken(req, res, next)` : Validate JWT token
- `registerUser(email, password, displayName)` : Create user account
- `getUserProfile(userId)` : Retrieve user data

Dependencies: Firebase Admin SDK, Express.js

4.2.2 Station Management Module

Purpose: Manage charging station data and availability

Components:

- `routes/stations.js` : Station endpoints

Endpoints:

- `GET /api/stations` : Get all stations (with filters)
- `GET /api/stations/:id` : Get station by ID
- `PUT /api/stations/:id/availability` : Update availability (IoT)

Key Functions:

- `getStations(latitude, longitude, radius)` : Query stations
- `updateStationAvailability(stationId, data)` : Update status
- `validateStationData(data)` : Input validation

Dependencies: Firebase Firestore, Express Validator

4.2.3 Reservation Management Module

Purpose: Handle reservation creation and management

Components:

- `routes/reservations.js` : Reservation endpoints

Endpoints:

- `GET /api/reservations` : Get user's reservations
- `POST /api/reservations` : Create reservation
- `PUT /api/reservations/:id` : Update reservation

Key Functions:

- `createReservation(userId, stationId, connectorId, times)` : Create reservation
- `checkAvailability(stationId, startTime, endTime)` : Validate slot
- `calculateCost(station, duration)` : Compute pricing

Dependencies: Firebase Firestore, Date/time utilities

4.2.4 Payment Processing Module

Purpose: Integrate with Stripe for secure payments

Components:

- `routes/payments.js` : Payment endpoints

Endpoints:

- `POST /api/payments/create-intent` : Create payment intent
- `POST /api/payments/confirm` : Confirm payment

Key Functions:

- `createPaymentIntent(amount, currency, metadata)` : Initialize payment
- `confirmPayment(paymentIntentId)` : Process transaction
- `handleWebhook(event)` : Process Stripe webhooks (future)

Dependencies: Stripe SDK, Firebase Firestore

4.2.5 IoT Integration Module

Purpose: Manage MQTT communication with IoT devices

Components:

- `routes/mqtt.js` : MQTT endpoints
- MQTT client instance

Endpoints:

- `POST /api/mqtt/publish/:stationId` : Publish command
- `GET /api/mqtt/status` : Get connection status

Key Functions:

- `connectMQTT(brokerUrl)` : Establish broker connection
- `subscribeToStations()` : Listen for status updates

- `publishCommand(stationId, command)` : Send device commands
- `syncToFirestore(topic, message)` : Update database

Dependencies: MQTT.js, Firebase Firestore

5. DATABASE DESIGN

5.1 Database Selection

Firebase Firestore is selected as the primary database for the following reasons:

1. **Real-time Synchronization:** Built-in support for real-time data updates
2. **Scalability:** Automatic scaling to handle large user bases
3. **Integration:** Seamless integration with Firebase Authentication and FCM
4. **Cloud-based:** No server management required
5. **Flexible Schema:** Document-based structure suitable for evolving requirements

5.2 Data Model

5.2.1 Users Collection

Document Structure:

```
{
  uid: string;           // Firebase Auth UID (Document ID)
  email: string;
  displayName: string;
  phoneNumber?: string;
  vehicleInfo?: {
    make: string;
    model: string;
    batteryCapacity: number; // kWh
    connectorType: string;   // Type1, Type2, CCS, CHAdeMO, Tesla
  };
  paymentMethods?: Array<{
    id: string;
    type: 'card' | 'bank_account';
    last4?: string;
    brand?: string;
    isDefault: boolean;
  }>;
  createdAt: Timestamp;
  updatedAt: Timestamp;
}
```

Indexes:

- `email` (single field index)
- `createdAt` (single field index)

5.2.2 ChargingStations Collection

Document Structure:

```

{
  id: string; // Document ID
  name: string;
  location: {
    latitude: number;
    longitude: number;
    address: string;
    city?: string;
    country?: string;
  };
  connectors: Array<{
    id: string;
    type: 'Type1' | 'Type2' | 'CCS' | 'CHAdemo' | 'Tesla';
    power: number; // kW
    available: boolean;
    currentStatus?: 'available' | 'occupied' | 'maintenance';
  }>;
  available: boolean; // Overall station availability
  occupied: number; // Number of occupied connectors
  total: number; // Total number of connectors
  pricing: {
    perKwh: number; // Price per kWh (currency unit)
    perMinute: number; // Price per minute (currency unit)
    currency: string; // e.g., 'TRY', 'USD', 'EUR'
  };
  amenities: Array<string>; // e.g., ['WiFi', 'Restroom', 'Cafe']
  rating: number; // Average rating (0-5)
  reviewCount: number;
  reviews: Array<{
    userId: string;
    userName: string;
    rating: number;
    comment: string;
    date: Timestamp;
  }>;
  operatingHours: {
    open: string; // e.g., "08:00"
    close: string; // e.g., "22:00"
    days: Array<string>; // e.g., ['Monday', 'Tuesday', ...]
  };
  operatorId?: string; // Station operator/administrator ID
  deviceId?: string; // MQTT device identifier
  createdAt: Timestamp;
  updatedAt: Timestamp;
}

```

Indexes:

- `location.latitude, location.longitude` (composite geospatial index)
- `available` (single field index)
- `operatorId` (single field index)

5.2.3 Reservations Collection

Document Structure:

```
{
  id: string;           // Document ID
  userId: string;        // Reference to users collection
  stationId: string;     // Reference to chargingStations collection
  connectorId: string;   // Specific connector identifier
  startTime: Timestamp;
  endTime: Timestamp;
  status: 'pending' | 'active' | 'completed' | 'cancelled';
  totalCost: number;
  paymentIntentId?: string; // Stripe payment intent ID
  paymentStatus: 'pending' | 'paid' | 'failed' | 'refunded';
  createdAt: Timestamp;
  updatedAt: Timestamp;
}
```

Indexes:

- `userId, status` (composite index)
- `stationId, startTime, endTime` (composite index)
- `status` (single field index)
- `createdAt` (single field index)

5.2.4 Payments Collection (Optional)

Document Structure:

```
{
  id: string;           // Document ID
  userId: string;
  reservationId: string;
  amount: number;
  currency: string;
  paymentIntentId: string; // Stripe payment intent ID
  status: 'succeeded' | 'failed' | 'pending';
  transactionId?: string;
  createdAt: Timestamp;
}
```

5.3 Data Relationships

1. **Users** → **Reservations**: One-to-many relationship (`userId` field)
2. **ChargingStations** → **Reservations**: One-to-many relationship (`stationId` field)
3. **Reservations** → **Payments**: One-to-one relationship (`reservationId` field)

5.4 Data Validation Rules

Firestore Security Rules ensure data integrity:

```

rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Users can read/write their own profile
    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }

    // Stations are readable by all, writable by admins
    match /chargingStations/{stationId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null &&

get(/databases/{(database)}/documents/users/{(request.auth.uid)}.data.role == 'admin';
    }

    // Users can read/write their own reservations
    match /reservations/{reservationId} {
      allow read, write: if request.auth != null &&
                           request.resource.data.userId == request.auth.uid;
    }
  }
}

```

6. API DESIGN

6.1 API Overview

The backend provides a RESTful API following REST principles:

- **Base URL:** `https://api.ecna.com/api` (production)
- **Development URL:** `http://localhost:3000/api`
- **Authentication:** JWT tokens in Authorization header
- **Content-Type:** `application/json`
- **Response Format:** JSON

6.2 Authentication Endpoints

6.2.1 Register User

Endpoint: `POST /api/auth/register`

Request Body:

```

{
  "email": "user@example.com",
  "password": "SecurePassword123!",
  "displayName": "John Doe"
}

```

Response (201 Created):

```
{
  "user": {
    "uid": "firebase-uid",
    "email": "user@example.com",
    "displayName": "John Doe"
  },
  "token": "jwt-token-string"
}
```

Error Responses:

- 400 Bad Request : Invalid input data
- 409 Conflict : Email already exists

6.2.2 Get Current User

Endpoint: GET /api/auth/me

Headers:

Authorization: Bearer <jwt-token>

Response (200 OK):

```
{
  "uid": "user-uid",
  "email": "user@example.com",
  "displayName": "John Doe",
  "phoneNumber": "+905551234567",
  "vehicleInfo": {
    "make": "Tesla",
    "model": "Model 3",
    "batteryCapacity": 75,
    "connectorType": "Type2"
  },
  "createdAt": "2025-01-15T10:30:00Z"
}
```

6.2.3 Update User Profile

Endpoint: PUT /api/auth/me

Headers:

Authorization: Bearer <jwt-token>

Request Body:

```
{
  "displayName": "John Updated",
  "phoneNumber": "+905559876543",
  "vehicleInfo": {
```

```
    "make": "Tesla",
    "model": "Model S",
    "batteryCapacity": 100,
    "connectorType": "Type2"
  }
}
```

6.3 Station Endpoints

6.3.1 Get All Stations

Endpoint: GET /api/stations

Query Parameters:

- latitude (optional): Filter by latitude
- longitude (optional): Filter by longitude
- radius (optional): Search radius in kilometers (default: 10)
- available (optional): Filter by availability (true/false)
- connectorType (optional): Filter by connector type

Response (200 OK):

```
{
  "stations": [
    {
      "id": "station-123",
      "name": "Downtown Charging Hub",
      "location": {
        "latitude": 37.7749,
        "longitude": -122.4194,
        "address": "123 Main St, San Francisco, CA"
      },
      "available": true,
      "occupied": 2,
      "total": 4,
      "pricing": {
        "perKwh": 2.50,
        "perMinute": 0.10,
        "currency": "USD"
      },
      "rating": 4.5,
      "reviewCount": 120
    }
  ],
  "total": 15
}
```

6.3.2 Get Station by ID

Endpoint: GET /api/stations/:id

Response (200 OK):

```
{
  "id": "station-123",
  "name": "Downtown Charging Hub",
  "location": {
    "latitude": 37.7749,
    "longitude": -122.4194,
    "address": "123 Main St, San Francisco, CA"
  },
  "connectors": [
    {
      "id": "conn-1",
      "type": "Type2",
      "power": 22,
      "available": true,
      "currentStatus": "available"
    },
    {
      "id": "conn-2",
      "type": "CCS",
      "power": 50,
      "available": false,
      "currentStatus": "occupied"
    }
  ],
  "available": true,
  "occupied": 2,
  "total": 4,
  "pricing": {
    "perKwh": 2.50,
    "perMinute": 0.10,
    "currency": "USD"
  },
  "amenities": ["WiFi", "Restroom", "Cafe"],
  "rating": 4.5,
  "reviewCount": 120,
  "reviews": [
    {
      "userId": "user-456",
      "userName": "Jane Smith",
      "rating": 5,
      "comment": "Great location and fast charging!",
      "date": "2025-01-10T14:30:00Z"
    }
  ],
  "operatingHours": {
    "open": "08:00",
    "close": "22:00",
    "days": ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
  }
}
```

6.3.3 Update Station Availability

Endpoint: PUT /api/stations/:id/availability

Headers:

```
Authorization: Bearer <jwt-token>
```

Request Body:

```
{
  "available": true,
  "occupied": 2,
  "connectors": [
    {
      "id": "conn-1",
      "available": true,
      "currentStatus": "available"
    },
    {
      "id": "conn-2",
      "available": false,
      "currentStatus": "occupied"
    }
  ]
}
```

Note: Typically called by IoT devices via MQTT webhook

6.4 Reservation Endpoints

6.4.1 Get User Reservations

Endpoint: GET /api/reservations

Headers:

```
Authorization: Bearer <jwt-token>
```

Query Parameters:

- `status` (optional): Filter by status (pending, active, completed, cancelled)

Response (200 OK):

```
{
  "reservations": [
    {
      "id": "res-789",
      "stationId": "station-123",
      "stationName": "Downtown Charging Hub",
      "connectorId": "conn-1",
      "startTime": "2025-01-20T10:00:00Z",
      "endTime": "2025-01-20T12:00:00Z",

```



```
    "status": "pending",
    "totalCost": 25.50,
    "paymentStatus": "pending",
    "createdAt": "2025-01-15T09:00:00Z"
  }
]
```

6.4.2 Create Reservation

Endpoint: POST /api/reservations

Headers:

Authorization: Bearer <jwt-token>

Request Body:

```
{
  "stationId": "station-123",
  "connectorId": "conn-1",
  "startTime": "2025-01-20T10:00:00Z",
  "endTime": "2025-01-20T12:00:00Z"
}
```

Response (201 Created):

```
{
  "id": "res-789",
  "userId": "user-456",
  "stationId": "station-123",
  "connectorId": "conn-1",
  "startTime": "2025-01-20T10:00:00Z",
  "endTime": "2025-01-20T12:00:00Z",
  "status": "pending",
  "totalCost": 25.50,
  "createdAt": "2025-01-15T09:00:00Z"
}
```

Error Responses:

- 400 Bad Request : Invalid time slot or connector unavailable
- 409 Conflict : Time slot already reserved

6.4.3 Update Reservation

Endpoint: PUT /api/reservations/:id

Headers:

Authorization: Bearer <jwt-token>

Request Body:

```
{
  "status": "cancelled"
}
```

6.5 Payment Endpoints

6.5.1 Create Payment Intent

Endpoint: POST /api/payments/create-intent

Headers:

```
Authorization: Bearer <jwt-token>
```

Request Body:

```
{
  "reservationId": "res-789",
  "amount": 2550,
  "currency": "USD"
}
```

Response (200 OK):

```
{
  "clientSecret": "pi_xxx_secret_xxx",
  "paymentIntentId": "pi_xxx"
}
```

6.5.2 Confirm Payment

Endpoint: POST /api/payments/confirm

Headers:

```
Authorization: Bearer <jwt-token>
```

Request Body:

```
{
  "paymentIntentId": "pi_xxx",
  "reservationId": "res-789"
}
```

Response (200 OK):

```
{
  "status": "succeeded",
  "transactionId": "ch_xxx",
}
```

```
"reservationId": "res-789"
}
```

6.6 MQTT Endpoints

6.6.1 Publish Command

Endpoint: POST /api/mqtt/publish/:stationId

Headers:

```
Authorization: Bearer <jwt-token>
```

Request Body:

```
{
  "command": "start_charging",
  "connectorId": "conn-1",
  "parameters": {
    "power": 22
  }
}
```

6.6.2 Get MQTT Status

Endpoint: GET /api/mqtt/status

Response (200 OK):

```
{
  "connected": true,
  "brokerUrl": "mqtt://broker.example.com:1883",
  "lastUpdate": "2025-01-15T10:30:00Z"
}
```

6.7 Error Response Format

All error responses follow this format:

```
{
  "error": {
    "message": "Error description",
    "code": "ERROR_CODE",
    "details": {}
  }
}
```

HTTP Status Codes:

- 200 OK : Successful request
- 201 Created : Resource created successfully
- 400 Bad Request : Invalid input

- 401 Unauthorized : Authentication required
 - 403 Forbidden : Insufficient permissions
 - 404 Not Found : Resource not found
 - 409 Conflict : Resource conflict
 - 500 Internal Server Error : Server error
-

7. USER INTERFACE DESIGN

7.1 Design Principles

The UI design follows these principles:

1. **User-Centered Design:** Intuitive navigation and clear information hierarchy
2. **Accessibility:** Support for screen readers and accessibility standards
3. **Responsive Design:** Adapts to different screen sizes and orientations
4. **Consistent Design Language:** Unified color scheme, typography, and components
5. **Performance Optimization:** Efficient rendering and minimal data loading

7.2 Screen Specifications

7.2.1 Authentication Screens

Login Screen:

- Email/password input fields
- "Remember me" option
- "Forgot password" link
- "Sign up" navigation link
- Primary action button: "Log In"

Register Screen:

- Email, password, display name inputs
- Password strength indicator
- Terms and conditions checkbox
- "Already have an account?" link
- Primary action button: "Sign Up"

7.2.2 Home Screen

Layout:

- Search bar at top
- Filter buttons (Available, Nearby, Fast Charging)
- Station list with cards showing:
 - Station name
 - Distance from user
 - Availability status
 - Rating and review count
 - Pricing information
- Pull-to-refresh functionality
- Navigation to Map view

7.2.3 Map Screen

Features:

- Interactive map with station markers
- Cluster markers for multiple stations
- User location indicator
- Marker info windows with quick details
- Bottom sheet with station details on marker tap
- Navigation button to station location
- Filter overlay

Map Controls:

- Zoom controls
- My location button
- Map type toggle (standard/satellite)

7.2.4 Station Details Screen**Sections:**

1. **Header:** Station name, rating, distance
2. **Availability:** Real-time status, occupied/total connectors
3. **Location:** Address, map preview, navigation button
4. **Connectors:** List of connectors with types, power, availability
5. **Pricing:** Per kWh and per minute rates
6. **Amenities:** Icon list of available amenities
7. **Reviews:** Scrollable review list with ratings
8. **Operating Hours:** Schedule display
9. **Action Button:** "Reserve Now" (if available)

7.2.5 Reservation Screen**Features:**

- Station information summary
- Connector selection (dropdown/list)
- Date and time pickers (start/end)
- Duration display
- Cost calculation preview
- Reservation summary
- Terms and conditions
- Primary action: "Confirm Reservation"

7.2.6 Payment Screen**Features:**

- Reservation summary
- Total amount display
- Payment method selection
- Card input fields (via Stripe SDK)
- Security badges/indicators
- Primary action: "Pay Now"
- Processing indicator

7.2.7 Profile Screen

Sections:

- User information display
- Vehicle information (editable)
- Payment methods management
- Reservation history
- Settings (notifications, language, etc.)
- Logout button

7.3 Navigation Structure

Navigation Pattern: Stack Navigation with Tab Navigation

```
App (Stack Navigator)
├─ Auth Stack
│  └─ Login Screen
│  └─ Register Screen
└─ Main Stack (Tab Navigator)
   ├─ Home Tab
   │  └─ Home Screen
   │  └─ Station Details Screen
   │  └─ Reservation Screen
   ├─ Map Tab
   │  └─ Map Screen
   │  └─ Station Details Screen
   ├─ Reservations Tab
   │  └─ Reservations List
   │  └─ Reservation Details
   └─ Profile Tab
      └─ Profile Screen
```

7.4 UI Components

Reusable Components:

- Button : Primary, secondary, text variants
- Input : Text input with validation
- Card : Station card, reservation card
- Badge : Status indicators, availability badges
- Rating : Star rating display
- LoadingIndicator : Spinner, skeleton loaders
- Modal : Confirmation dialogs, info modals
- BottomSheet : Station details, filters

7.5 Color Scheme

Primary Colors:

- Primary: #00C853 (Green - EV theme)
- Primary Dark: #00A043
- Primary Light: #5EFC82

Secondary Colors:

- Secondary: #2196F3 (Blue)

- Accent: #FF9800 (Orange)

Status Colors:

- Success: #4CAF50
- Warning: #FF9800
- Error: #F44336
- Info: #2196F3

Neutral Colors:

- Background: #F5F5F5
 - Surface: #FFFFFF
 - Text Primary: #212121
 - Text Secondary: #757575
 - Divider: #BDBDBD
-

8. SECURITY DESIGN

8.1 Authentication and Authorization

8.1.1 User Authentication

Firestore Authentication:

- Email/password authentication
- JWT token generation
- Token expiration: 1 hour (default)
- Token refresh mechanism
- Password hashing: Firestore managed (bcrypt-based)

Token Management:

- Tokens stored securely on device (AsyncStorage/Keychain)
- Automatic token refresh before expiration
- Token validation on each API request

8.1.2 Authorization

Role-Based Access Control (RBAC):

- **User Role:** Standard EV user (default)
- **Admin Role:** Station operators and administrators

Permission Matrix:

- Users can read stations, create reservations, update own profile
- Admins can create/update stations, view analytics, manage reservations

8.2 Data Security

8.2.1 Data Encryption

In Transit:

- HTTPS/TLS 1.2+ for all API communications
- MQTT over TLS (MQTTS) for IoT communications
- Certificate pinning (mobile apps)

At Rest:

- Firebase Firestore: Encrypted by default
- Sensitive data (payment info): Stripe handles encryption
- Local storage: Encrypted storage for tokens

8.2.2 Input Validation

Backend Validation:

- Express Validator for request validation
- Schema validation for data models
- SQL injection prevention (NoSQL, but sanitization applied)
- XSS prevention through input sanitization

Frontend Validation:

- Client-side validation for user experience
- Server-side validation as security layer (never trust client)

8.3 API Security

8.3.1 Rate Limiting

Implementation:

- Express Rate Limit middleware
- Limits: 100 requests per 15 minutes per IP
- Stricter limits for authentication endpoints: 5 requests per 15 minutes

Purpose: Prevent brute force attacks and API abuse

8.3.2 Security Headers

Helmet.js Configuration:

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: DENY`
- `X-XSS-Protection: 1; mode=block`
- `Strict-Transport-Security: max-age=31536000`
- `Content-Security-Policy` : Appropriate directives

8.3.3 CORS Configuration

Settings:

- Whitelist specific origins (production)
- Credentials: true (for cookies/auth)
- Allowed methods: GET, POST, PUT, DELETE
- Allowed headers: Content-Type, Authorization

8.4 Payment Security

8.4.1 Stripe Integration

PCI Compliance:

- No card data stored on servers
- Stripe Elements/SDK handles card input
- Payment intents created server-side

- Client secrets transmitted securely

Payment Flow Security:

1. Client requests payment intent from backend
2. Backend creates intent with Stripe API
3. Client receives client secret (non-sensitive)
4. Client confirms payment with Stripe SDK
5. Backend verifies payment status via webhook

8.5 IoT Security

8.5.1 MQTT Security

Authentication:

- Username/password authentication for MQTT broker
- Certificate-based authentication (production)
- Topic access control (ACL)

Topic Structure:

- Stations publish to: `stations/{stationId}/status`
- Backend publishes commands to: `stations/{stationId}/commands`
- Topic permissions restrict device access

Message Security:

- Encrypted payloads (optional, for sensitive commands)
- Message authentication codes (MAC) for integrity

8.6 Privacy Protection

8.6.1 Data Privacy

User Data:

- Minimal data collection (only necessary fields)
- User consent for data processing
- Right to delete account and data (GDPR compliance)

Location Data:

- Location used only for station discovery
- Not stored permanently without consent
- Optional location services

8.6.2 Data Retention

- User data: Retained while account is active
- Reservation data: Retained for 2 years (accounting)
- Payment data: Handled by Stripe (their retention policy)
- Logs: Retained for 30 days (security monitoring)

9. IoT INTEGRATION DESIGN

9.1 IoT Architecture

The IoT integration enables real-time communication between charging stations and the application backend.

Communication Flow:

IoT Device (Station) ↔ MQTT Broker ↔ Backend Server ↔ Firestore ↔ Mobile App

9.2 MQTT Protocol

9.2.1 Protocol Selection

MQTT (Message Queuing Telemetry Transport) is chosen because:

- Lightweight protocol suitable for IoT devices
- Publish-subscribe pattern for real-time updates
- Low bandwidth requirements
- Reliable message delivery (QoS levels)
- Wide industry adoption

9.2.2 MQTT Topics

Topic Structure:

```
stations/{stationId}/status          # Station publishes status updates
stations/{stationId}/commands        # Backend publishes commands
stations/{stationId}/connectors/{connectorId}/status # Per-connector status
stations/{stationId}/connectors/{connectorId}/commands # Per-connector commands
```

Message Format (JSON):

```
{
  "timestamp": "2025-01-15T10:30:00Z",
  "stationId": "station-123",
  "available": true,
  "occupied": 2,
  "total": 4,
  "connectors": [
    {
      "id": "conn-1",
      "available": true,
      "currentStatus": "available",
      "power": 22,
      "energyDelivered": 15.5
    }
  ]
}
```

9.3 Backend MQTT Integration

9.3.1 MQTT Client

Implementation:

- MQTT.js library for Node.js
- Persistent connection to broker
- Automatic reconnection on disconnect
- QoS Level 1 (at least once delivery)

Subscriptions:

- Subscribe to all station status topics: `stations+/status`
- Subscribe to connector topics: `stations+/connectors+/status`

Publishing:

- Commands published to device topics
- QoS Level 1 for reliability

9.3.2 Firestore Synchronization**Sync Logic:**

1. Receive MQTT message from station
2. Validate message structure
3. Update Firestore document atomically
4. Trigger real-time listeners (mobile apps receive update)
5. Log update for audit trail

Conflict Resolution:

- Last-write-wins strategy (acceptable for status updates)
- Timestamp-based ordering

9.4 Frontend Real-time Updates**9.4.1 MQTT Service****Implementation:**

- MQTT.js in React Native (via react-native-mqtt or WebSocket bridge)
- Alternative: WebSocket connection to backend (backend forwards MQTT messages)

Update Strategy:

1. Connect to MQTT broker (or WebSocket)
2. Subscribe to relevant station topics
3. Update local state on message receipt
4. Re-render UI components

Optimization:

- Subscribe only to visible stations (map view)
- Unsubscribe when leaving screen
- Batch updates to reduce re-renders

9.5 Command Publishing**9.5.1 Command Types****Available Commands:**

- `start_charging` : Initiate charging session
- `stop_charging` : Terminate charging session
- `update_status` : Request status update
- `reset` : Reset device (admin only)

Command Format:

```
{
  "command": "start_charging",
  "connectorId": "conn-1",
  "parameters": {
    "power": 22,
    "duration": 7200
  },
  "userId": "user-456",
  "reservationId": "res-789",
  "timestamp": "2025-01-15T10:30:00Z"
}
```

9.5.2 Command Flow

1. User initiates action (e.g., start charging)
2. Frontend calls backend API: `POST /api/mqtt/publish/:stationId`
3. Backend validates user permissions and reservation
4. Backend publishes command to MQTT topic
5. IoT device receives command and executes
6. Device publishes status update
7. Backend syncs to Firestore
8. Frontend receives real-time update

9.6 Error Handling

Connection Failures:

- Automatic reconnection with exponential backoff
- Fallback to polling if MQTT unavailable
- User notification of connectivity issues

Message Failures:

- Retry mechanism for failed commands
- Timeout handling (commands expire after 30 seconds)
- Error logging and monitoring

10. PERFORMANCE REQUIREMENTS

10.1 Response Time Requirements

API Response Times:

- Authentication endpoints: < 500ms
- Station list queries: < 1s
- Station details: < 500ms
- Reservation creation: < 1s
- Payment processing: < 2s (excluding Stripe processing)
- Real-time updates: < 100ms (MQTT latency)

Mobile App Performance:

- Screen load time: < 2s
- Map rendering: < 3s

- Image loading: Progressive/lazy loading
- Smooth scrolling: 60 FPS

10.2 Scalability Requirements

User Capacity:

- Support 10,000 concurrent users
- Handle 1,000 requests per second (API)
- Database: Scale to 1 million users, 100,000 stations

Data Growth:

- Reservations: 1 million per year
- Real-time updates: 100 updates per second (peak)
- Storage: Efficient data archiving strategy

10.3 Availability Requirements

Uptime:

- System availability: 99.5% (43.8 hours downtime/year)
- Critical services (payments, reservations): 99.9%

Redundancy:

- Database: Firebase Firestore (Google Cloud redundancy)
- Backend: Multiple server instances with load balancing
- MQTT Broker: High-availability cluster

10.4 Optimization Strategies

10.4.1 Database Optimization

- Indexes on frequently queried fields
- Pagination for large result sets
- Caching frequently accessed data (Redis - future)
- Data aggregation for analytics

10.4.2 API Optimization

- Response compression (gzip)
- Request batching where possible
- GraphQL consideration (future) for reduced over-fetching
- CDN for static assets

10.4.3 Mobile App Optimization

- Code splitting and lazy loading
- Image optimization and caching
- Offline support (cached data)
- Bundle size optimization (tree shaking)

11. DEPLOYMENT ARCHITECTURE

11.1 Deployment Overview

The system is deployed across multiple cloud services:

- **Mobile Apps:** iOS App Store, Google Play Store
- **Backend API:** Cloud platform (AWS, Google Cloud, or Heroku)
- **Database:** Firebase Firestore (Google Cloud)
- **Authentication:** Firebase Authentication
- **MQTT Broker:** Cloud MQTT service (e.g., CloudMQTT, AWS IoT Core)
- **CDN:** CloudFront or Cloud CDN (for static assets)

11.2 Backend Deployment

11.2.1 Server Configuration

Recommended Platform: Node.js hosting (AWS EC2, Google Cloud Run, Heroku)

Environment Variables:

```
NODE_ENV=production
PORT=3000
FRONTEND_URL=https://app.ecna.com
FIREBASE_SERVICE_ACCOUNT=<json>
STRIPE_SECRET_KEY=<key>
MQTT_BROKER_URL=mqtts://broker.ecna.com:8883
MQTT_USERNAME=<username>
MQTT_PASSWORD=<password>
JWT_SECRET=<secret>
```

Process Management: PM2 or systemd

Load Balancing: Multiple instances behind load balancer

11.2.2 CI/CD Pipeline

Stages:

1. **Build:** Install dependencies, run tests
2. **Test:** Unit tests, integration tests
3. **Deploy:** Deploy to staging environment
4. **Integration Tests:** E2E tests on staging
5. **Deploy Production:** Manual approval, deploy to production
6. **Monitor:** Health checks and monitoring

Tools: GitHub Actions, GitLab CI, or Jenkins

11.3 Mobile App Deployment

11.3.1 Build Process

React Native (Expo):

1. Configure app.json with production settings
2. Build iOS: `eas build --platform ios`
3. Build Android: `eas build --platform android`
4. Submit to app stores

App Store Requirements:

- iOS: App Store Connect submission
- Android: Google Play Console submission

- Privacy policy URL
- App icons and screenshots
- Store listings

11.3.2 Update Strategy

Over-The-Air (OTA) Updates:

- Expo Updates for JavaScript bundle updates
- Native updates require app store submission
- Gradual rollout strategy

11.4 Database Deployment

Firestore:

- No deployment required (managed service)
- Configuration via Firebase Console
- Security rules deployed via Firebase CLI
- Backup strategy: Automated daily backups

11.5 Monitoring and Logging

11.5.1 Application Monitoring

Metrics:

- API response times
- Error rates
- Request counts
- Resource utilization (CPU, memory)

Tools:

- Application Performance Monitoring (APM): New Relic, Datadog, or Firebase Performance
- Error tracking: Sentry
- Logging: Winston, Bunyan, or Cloud Logging

11.5.2 Health Checks

Endpoints:

- GET /health : Basic health check
- GET /health/detailed : Database, MQTT connectivity

Monitoring:

- Uptime monitoring: Pingdom, UptimeRobot
- Alerting: Email, SMS, Slack notifications

12. TESTING STRATEGY

12.1 Testing Levels

12.1.1 Unit Testing

Scope: Individual functions and components

Frontend:

- React Native Testing Library
- Jest for JavaScript/TypeScript
- Component testing (snapshot tests)
- Service function testing

Backend:

- Jest or Mocha for Node.js
- Route handler testing
- Utility function testing
- Mock dependencies (Firebase, Stripe, MQTT)

Coverage Target: 70% code coverage

12.1.2 Integration Testing

Scope: Module interactions and API endpoints

Backend API Testing:

- Supertest for Express endpoints
- Test database (Firebase Emulator)
- Mock external services (Stripe test mode, MQTT mock)
- Test authentication flows
- Test payment flows

Frontend Integration:

- Test navigation flows
- Test API service calls
- Test Firebase integration
- Mock API responses

12.1.3 End-to-End Testing

Scope: Complete user workflows

Tools: Detox (React Native E2E testing)

Test Scenarios:

1. User registration and login
2. Search for stations and view details
3. Create reservation
4. Complete payment
5. View reservation history
6. Receive real-time updates

12.1.4 Performance Testing

Load Testing:

- Tools: Artillery, k6, or Apache JMeter
- Test API endpoints under load
- Measure response times and throughput
- Identify bottlenecks

Stress Testing:

- Test system limits
- Database query performance
- Concurrent user simulation

12.2 Test Data Management

Test Databases:

- Firebase Emulator Suite for development
- Separate test Firebase project for integration tests
- Seed data scripts for consistent test data

Mock Data:

- Station data fixtures
- User test accounts
- Payment test cards (Stripe test mode)

12.3 Test Automation

Continuous Integration:

- Run tests on every commit
- Run tests before deployment
- Fail build on test failures

Test Reports:

- Generate coverage reports
 - Test result summaries
 - Visual regression testing (screenshots)
-

13. REFERENCES

13.1 Academic References

1. International Energy Agency (IEA). (2023). Global EV Outlook 2023: Catching up with climate ambitions. Paris: IEA Publications.
2. Azzopardi, B., & Gan, C. K. (2022). Smart Charging Infrastructure for Electric Vehicles: Integration with Smart Grids and Renewable Energy Systems. Renewable and Sustainable Energy Reviews, 161, 112357.
3. Shukla, A., & Jain, S. (2021). Cloud-Based IoT Architecture for Electric Vehicle Charging Network Management. IEEE Access, 9, 77154–77165.
4. Mahmoud, M. S., & Nasir, M. T. (2020). Cybersecurity and Privacy in Smart Grids: Challenges and Solutions. IEEE Systems Journal, 14(3), 4505–4516.
5. Ghosh, A. (2023). Integration of Electric Vehicles and Renewable Energy Sources in Smart Cities: A Review. Energy Reports, 9, 3152–3166.

13.2 Technical Documentation

1. React Native Documentation. (2024). Retrieved from <https://reactnative.dev/docs/getting-started>
2. Firebase Documentation. (2024). Retrieved from <https://firebase.google.com/docs>

3. Stripe API Documentation. (2024). Retrieved from <https://stripe.com/docs/api>

4. MQTT Protocol Specification. (2019). OASIS Standard. Retrieved from <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

5. Express.js Documentation. (2024). Retrieved from <https://expressjs.com/>

6. Node.js Documentation. (2024). Retrieved from <https://nodejs.org/docs>

13.3 Standards and Protocols

1. OAuth 2.0 Authorization Framework. (2012). RFC 6749. IETF.

2. JSON Web Token (JWT). (2015). RFC 7519. IETF.

3. Representational State Transfer (REST). (2000). Architectural Style. Fielding, R. T.

4. Message Queuing Telemetry Transport (MQTT). (2019). OASIS Standard v5.0.

5. Transport Layer Security (TLS). (2018). RFC 8446. IETF.

APPENDIX A: GLOSSARY

- **API:** Application Programming Interface
- **CDN:** Content Delivery Network
- **CI/CD:** Continuous Integration/Continuous Deployment
- **E2E:** End-to-End
- **FCM:** Firebase Cloud Messaging
- **HTTPS:** Hypertext Transfer Protocol Secure
- **IoT:** Internet of Things
- **JWT:** JSON Web Token
- **MQTT:** Message Queuing Telemetry Transport
- **OTA:** Over-The-Air
- **PCI:** Payment Card Industry
- **QoS:** Quality of Service
- **REST:** Representational State Transfer
- **SDK:** Software Development Kit
- **TLS:** Transport Layer Security

APPENDIX B: ACRONYMS

- **ECNA:** EV Charging Network App
- **EV:** Electric Vehicle
- **SDD:** Software Design Document
- **RBAC:** Role-Based Access Control
- **GDPR:** General Data Protection Regulation
- **XSS:** Cross-Site Scripting
- **ACL:** Access Control List
- **MAC:** Message Authentication Code
- **APM:** Application Performance Monitoring